

Week8

Task1:

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int *ptr1, *ptr2, *ptr3;
    int i;
    ptr1 = (int *)malloc(5 * sizeof(int));

    if (ptr1 == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        ptr1[i] = i + 1;

    printf("malloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", ptr1[i]);
    printf("\n");
    ptr2 = (int *)calloc(5, sizeof(int));

    if (ptr2 == NULL)
    {
        printf("calloc failed\n");
        free(ptr1);
        return 1;
    }
    printf("calloc: ");
    for (i = 0; i < 5; i++)
        printf("%d ", ptr2[i]);
    printf("\n");
    ptr1 = (int *)realloc(ptr1, 10 * sizeof(int));

    if (ptr1 == NULL)
    {
        printf("realloc failed\n");
```

```

    free(ptr2);
    return 1;
}

for (i = 5; i < 10; i++)
    ptr1[i] = i + 1;

printf("After realloc: ");
for (i = 0; i < 10; i++)
    printf("%d ", ptr1[i]);
printf("\n");
ptr3 = (int *)malloc(10 * sizeof(int));
if (ptr3 == NULL)
{
    printf("malloc failed\n");
    free(ptr1);
    free(ptr2);
    return 1;
}
for (i = 0; i < 10; i++)
    ptr3[i] = i + 1;
memmove(ptr3 + 2, ptr3, 8 * sizeof(int));
printf("After memmove: ");
for (i = 0; i < 10; i++)
    printf("%d ", ptr3[i]);
printf("\n");
free(ptr1);
free(ptr2);
free(ptr3);

printf("\nAll allocated memory freed successfully.\n");

return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./memory
malloc: 1 2 3 4 5
calloc: 0 0 0 0 0
After realloc: 1 2 3 4 5 6 7 8 9 10
After memmove: 1 2 1 2 3 4 5 6 7 8

All allocated memory freed successfully.
root@DESKTOP-VMN932K:~#

```

Task2:

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int *data = malloc(5 * sizeof(int));

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        data[i] = i + 1;

    printf("Parent before fork:\n");
    printf("Data address = %p\n", (void *)data);
    printf("Data[0] = %d\n\n", data[0]);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        /* Child process */
        printf("Child before modification:\n");
        printf("Data address = %p\n", (void *)data);
        printf("Data[0] = %d\n\n", data[0]);

        /* Modifying data triggers Copy-on-Write */
        data[0] = 100;
    }
}
```

```

        printf("Child after modification:\n");
        printf("Data address = %p\n", (void *)data);
        printf("Data[0] = %d\n\n", data[0]);

        free(data);
        exit(0);
    }
    else
    {
        /* Parent process */
        wait(NULL);

        printf("Parent after child modification:\n");
        printf("Data address = %p\n", (void *)data);
        printf("Data[0] = %d\n", data[0]);

        free(data);
    }

    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./fork2
Parent before fork:
Data address = 0x5fd3c9b592a0
Data[0] = 1

Child before modification:
Data address = 0x5fd3c9b592a0
Data[0] = 1

Child after modification:
Data address = 0x5fd3c9b592a0
Data[0] = 100

Parent after child modification:
Data address = 0x5fd3c9b592a0
Data[0] = 1
root@DESKTOP-VMN932K:~#

```